

Informe del examen: CRUD con Supabase en Next.js

Proyecto: examen-reynaldo

Objetivo: terminar la parte práctica del examen creando una app Next.js que gestione cursos en Supabase.

Fecha: 22 de mayo de 2026

1. Resumen de lo que se pidió

La tarea pedía crear una aplicación Next.js conectada a Supabase para gestionar una tabla llamada `cursos`.

La tabla debía tener estos campos:

- `id`
- `nombre`
- `modalidad`, con valores `presencial` u `online`
- `horas`
- `created_at`
- `updated_at`, añadido para la segunda parte

Tarea 1: CRUD con Supabase

- Mostrar el listado completo de cursos en un Client Component.
- Usar `useEffect` y `useState`.
- Añadir un formulario para insertar cursos con `insert()`.
- Añadir un botón **Eliminar** en cada curso usando `delete().eq('id', id)`.
- Leer las credenciales de Supabase desde `.env.local`.

Tarea 2: Función y trigger en PostgreSQL

- Añadir el campo `updated_at`.
- Crear la función `set_updated_at()`.
- Crear un trigger `BEFORE UPDATE`.
- Comprobar que al actualizar un curso cambia `updated_at`.

2. Archivos creados o modificados

- `app/page.jsx`: pantalla principal con listado, formulario e inserción/eliminación de cursos.

- `utils/supabase/client.js` : cliente de Supabase leyendo variables de entorno.
- `supabase/snippets/cursos_updated_at.sql` : SQL para ejecutar en Supabase SQL Editor.
- `supabase/migrations/20260522121000_create_cursos.sql` : migración formal del proyecto.
- `PASOS_EXAMEN.md` : guía de estudio paso a paso.
- `informe-examen-supabase-next.html` : versión HTML de este informe.
- `informe-examen-supabase-next.pdf` : PDF final exportado.

3. Paso a paso realizado en el proyecto

Paso 1: Revisar el proyecto

Primero se revisó la estructura del proyecto y la versión instalada de Next.js. El proyecto usa:

- `next 16.2.6`
- `react 19.2.4`
- `@supabase/supabase-js 2.106.1`

También se leyó la documentación local de Next dentro de `node_modules/next/dist/docs/` , porque el archivo `AGENTS.md` avisaba que esta versión de Next podía tener cambios importantes.

Paso 2: Corregir la ruta del cliente Supabase

La página importaba:

```
import { supabase } from '@utils/supabase/client'
```

Pero el proyecto tenía una carpeta escrita como `utlis` , no `utils` . Para que la importación funcionara correctamente, se creó:

```
utils/supabase/client.js
```

Paso 3: Crear el cliente de Supabase

El cliente se creó con `createClient` . Este archivo lee las credenciales de `.env.local` :

```
import { createClient } from '@supabase/supabase-js'

const supabaseUrl = process.env.NEXT_PUBLIC_SUPABASE_URL
const supabaseAnonKey = process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY

if (!supabaseUrl || !supabaseAnonKey) {
  throw new Error('Faltan NEXT_PUBLIC_SUPABASE_URL o NEXT_PUBLIC_SUPABASE_ANON_KEY')
}

export const supabase = createClient(supabaseUrl, supabaseAnonKey)
```

Esto permite usar `supabase` desde la página principal para hacer `select` , `insert` y `delete` .

Importante: el archivo `.env.local` actual apunta a Supabase local:

```
NEXT_PUBLIC_SUPABASE_URL=http://127.0.0.1:54321
```

Si se usa un proyecto en **supabase.com**, hay que cambiar la URL y la anon key por las credenciales del proyecto remoto.

Paso 4: Crear la interfaz en `app/page.jsx`

La página se marcó como Client Component usando:

```
'use client'
```

Esto es necesario porque la página usa estado, eventos y `useEffect` .

Paso 5: Crear los estados

Se usaron estados para guardar los datos y controlar la pantalla:

```
const [cursos, setCursos] = useState([])
const [nombre, setNombre] = useState('')
const [modalidad, setModalidad] = useState('presencial')
const [horas, setHoras] = useState('')
const [cargando, setCargando] = useState(true)
const [guardando, setGuardando] = useState(false)
const [mensaje, setMensaje] = useState('')
```

- `cursos` : lista de cursos que viene desde Supabase.
- `nombre` , `modalidad` , `horas` : valores del formulario.
- `cargando` : indica si se están cargando los datos.
- `guardando` : indica si se está insertando un curso.
- `mensaje` : muestra errores o confirmaciones.

Paso 6: Cargar cursos con `useEffect`

Al abrir la página, se ejecuta `cargarCursos()` :

```
useEffect(() => {
  cargarCursos()
}, [])
```

El array vacío `[]` significa que se ejecuta una vez al cargar la página.

Paso 7: Hacer el READ con Supabase

La función `cargarCursos` consulta la tabla `cursos` :

```
const { data, error } = await supabase
  .from('cursos')
  .select('id, nombre, modalidad, horas, created_at, updated_at')
  .order('created_at', { ascending: false })
```

Si hay error, se muestra un mensaje. Si todo sale bien, se guardan los cursos en el estado `cursos` .

Paso 8: Insertar cursos con insert()

Cuando se envía el formulario, se ejecuta `crearCurso` . Primero se evita que la página se recargue:

```
event.preventDefault()
```

Después se validan los datos y se inserta la fila:

```
const { error } = await supabase.from('cursos').insert({
  nombre: nombre.trim(),
  modalidad,
  horas: horasNumero,
})
```

No se envía `id` ni `created_at` porque PostgreSQL los genera automáticamente.

Paso 9: Eliminar cursos con delete().eq()

Cada fila de la tabla tiene un botón **Eliminar**. Al pulsarlo se llama:

```
eliminarCurso(curso.id)
```

La función borra solo el curso que tenga ese `id` :

```
const { error } = await supabase
  .from('cursos')
  .delete()
  .eq('id', id)
```

La parte `.eq('id', id)` es fundamental porque indica la condición del borrado.

Paso 10: Mostrar la tabla

Se recorre el array `cursos` con `map` :

```
{cursos.map((curso) => (
  <tr key={curso.id}>
    ...
```

```
</tr>  
)}
```

Por cada curso se crea una fila con sus datos: id, nombre, modalidad, horas, fecha de creación, fecha de actualización y botón de eliminar.

4. SQL de la base de datos

Para crear la tabla y el trigger se preparó este SQL:

```
create table if not exists cursos (  
  id bigint generated by default as identity primary key,  
  nombre text not null,  
  modalidad text not null check (modalidad in ('presencial', 'online')),  
  horas integer not null check (horas > 0),  
  created_at timestamptz not null default now()  
);  
  
alter table cursos  
add column if not exists updated_at timestamptz;  
  
create or replace function set_updated_at()  
returns trigger as $$  
begin  
  new.updated_at = now();  
  return new;  
end;  
$$ language plpgsql;  
  
drop trigger if exists cursos_set_updated_at on cursos;  
  
create trigger cursos_set_updated_at  
before update on cursos  
for each row  
execute function set_updated_at();
```

Explicación del SQL

1. `create table if not exists cursos` : crea la tabla si todavía no existe.
2. `id bigint generated by default as identity` : crea un id automático.
3. `modalidad ... check` : obliga a que modalidad sea `presencial` u `online`.
4. `created_at timestamptz default now()` : guarda la fecha de creación automáticamente.
5. `alter table ... updated_at` : añade el campo de última modificación.
6. `set_updated_at()` : función que pone `updated_at` con la fecha actual.
7. `trigger before update` : ejecuta la función antes de cada actualización.

5. Cómo crear la tabla si no aparece en Supabase

Si en Supabase no se ve ninguna tabla, significa que el SQL todavía no se ha ejecutado en esa base de datos.

1. Abre Supabase.
2. Entra en tu proyecto.

3. Ve a **SQL Editor**.
4. Abre el archivo `supabase/snippets/cursos_updated_at.sql` en este proyecto.
5. Copia su contenido.
6. Pégallo en SQL Editor.
7. Ejecuta el SQL.
8. Ve a **Table Editor** y comprueba que existe la tabla `cursos`.

Muy importante: si la app usa Supabase local pero tú miras Supabase remoto, no verás la tabla en el sitio donde esperas.

Hay que comprobar que `.env.local` apunta exactamente a la base que estás mirando.

6. Cómo comprobar que `updated_at` funciona

En el archivo `supabase/snippets/cursos_updated_at.sql` hay una comprobación final:

```
with curso_demo as (  
  insert into cursos (nombre, modalidad, horas)  
  values ('Curso de prueba', 'online', 12)  
  returning id  
)  
update cursos  
set nombre = 'Curso de prueba actualizado'  
from curso_demo  
where cursos.id = curso_demo.id  
returning cursos.id, cursos.nombre, cursos.updated_at;
```

Ese bloque hace tres cosas:

1. Inserta un curso de prueba.
2. Actualiza el nombre del curso.
3. Devuelve `updated_at`.

Si `updated_at` aparece con una fecha, el trigger funciona.

7. Verificaciones realizadas

Se ejecutaron estas comprobaciones:

- `npm run build` : compiló correctamente.
- `npm run lint` : no devolvió errores.
- La app respondió con código HTTP `200` en `http://127.0.0.1:3000`.

8. Orden recomendado para repetirlo en clase

1. Crear el proyecto Next.js.
2. Instalar `@supabase/supabase-js`.
3. Crear o revisar `.env.local`.
4. Crear `utils/supabase/client.js`.
5. Crear la tabla `cursos` en Supabase con SQL Editor.
6. Crear la página como Client Component con `'use client'`.
7. Crear estados con `useState`.
8. Cargar cursos con `useEffect`.
9. Mostrar la tabla con `map`.
10. Crear el formulario.
11. Insertar con `insert()`.
12. Eliminar con `delete().eq('id', id)`.
13. Añadir `updated_at`.
14. Crear función `set_updated_at()`.
15. Crear trigger `BEFORE UPDATE`.
16. Probar la app.

9. Resumen final para estudiar

React: usa `useState` para guardar datos y `useEffect` para cargar datos al abrir la página.

Supabase: usa `.from('cursos')` para indicar la tabla.

Leer: `select()`.

Crear: `insert()`.

Eliminar: `delete().eq('id', id)`.

PostgreSQL: usa una función y un trigger para actualizar `updated_at` automáticamente.